

Advanced Statistical Methods: Coursework Report

Author: Ilya Kogan
Data Intensive Science
DAMTP, University of Cambridge



Word Count: 2984

April 7, 2025

1 Introduction

This coursework project covers the recreation of Bayesian Inference analysis in [1] applied to de-mystifying the purpose of the Antikythera mechanism by inferencing about the number of holes the mechanism may have contained. In the process we define 2 models: one simpler - with 1 type of error and another more complicated - with 2 types of errors. Using numpyro, a probabilistic programming language (PPL), we infer about posteriors, then identify a better model of the two and test whether it is a good model in general by performing a Posterior Predictive Check.

2 Data Visualization and Exploration

Firstly, we understand the data we will be working with which was obtained from [2]. It contains 2 images, a README file, and a csv file which contains the data: mean (x,y) coordinates of the center of 81 holes based on a few measurements (so that the standard deviation of its location is less than the image resolution of 0.05mm), holes' Section IDs and hole numbers as defined in [3], and Inter-hole distance which was only calculated between points of not cracked and not separated segments.

After confirming that measurements of location are present for all holes, we plot them to visualize as shown in 1. Since we aren't using inter-hole distances in our analysis, it is not problematic that not all holes have this information.

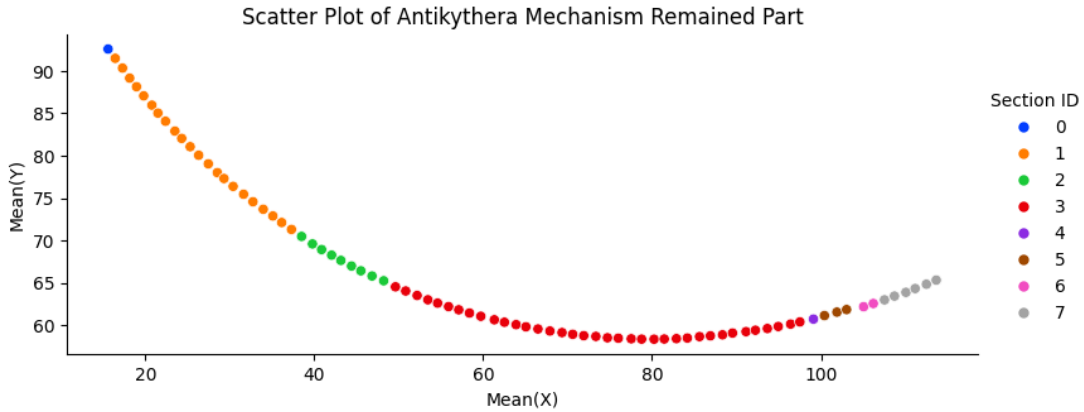


Figure 1: Visualization of hole locations of the remaining part of Antikythera Mechanism differentiated by Section IDs as measured by Budiselic et al[3]. Illustrates that Section 0 and Section 4 each have only 1 hole in them which means that they can't constrain parameters of our model in any way while Sections 3 and 1 are the largest and should impact our decision the most.

3 Model Building

Secondly, we build the model of the Antikythera Mechanism based on its remaining part. We assume that the Antikythera Mechanism was a circle of radius r containing N holes. For each of 8 remained section arcs, as defined in [3], we have 3 parameters: (x_{0j}, y_{0j}) - centers of the circle and α_j - angular position of the first hole of the full circle. Since we believe that our section arcs come from the same circle, we expect these α_j and (x_{0j}, y_{0j}) to be similar. The reason for modeling each section separately (with 24 parameters overall) rather than modeling with 1 circle (hence, 3 parameters) is to account for the cracks and separations in the remaining part. Angular position of the i th hole in the j th section with respect to its arc-centre is $\phi_{ij} = 2\pi \frac{i-1}{N} + \alpha_j$. [1] Thus, the position of i -th hole in the j -th section in Cartesian coordinates is given by $p_{ij} = r * (\cos\phi_{ij}, \sin\phi_{ij}) + (x_{0j}, y_{0j})$. We note that this model is based on an assumption that all points lie in the (x,y) plane, meaning that there is no vertical displacement and that the N holes are arranged regularly around a circle of radius r with broken sections of the ring allowed to be misaligned by translations and rotations in the plane. [1]

We define a function, *holes.locations* which outputs coordinates of 81 holes based on parameters that define a circle composed of 8 section arcs as described above. Each section contains number of points as defined in [3] and shown in 1 and there are 27 parameters in our model: N , r , σ , (x_{0j}, y_{0j}) and α_j for each section arc j , $0 \leq j \leq 7$. Since Sections 0 and 4 contain only one point each, their presence or absence in our analysis is not going to change virtually anything.

Section ID	0	1	2	3	4	5	6	7
Number of holes	1	22	9	37	1	3	2	6

Table 1: Table illustrating number of holes in each section which were defined by cracks and separations of the excavated part of the Antikythera Mechanism.

Our model is built using numpyro, a probabilistic programming language, where each variable is treated as a random variable. This approach provides a few advantages: the model is generational and can be used for both predictions and inference, plus numpyro contains an internal inference engine which is very useful and easy-to-run for inference tasks. As in [1], we choose uniform, or uninformative, priors for our parameters. Based on Budiselic et al, we choose slightly wider limits for uniform distribution of N : from 340 to 370 and for now N is regarded as a real number rather than an integer. For r , our limits are 50 and 100; for σ , 0 and 1; for each x-coordinate of the circle center, 0 to 150, for each y-coordinate: 50 to 200; and from -180 to 180 for each α_j as shown in 2. For each circle j , we have independent samples from our prior distributions. The final step of our model involves sampling Cartesian coordinates for 81 points from a Multivariate Normal distribution, where the mean is given by the model and the standard deviation is defined by a diagonal covariance matrix where each diagonal element is σ^2 . This sampling is based on parameters drawn from priors and used in the *holes.locations* function, as described earlier, and

takes into account the observed data. By using an isotropic Multivariate Normal distribution, we incorporate the Gaussian likelihood function for each hole location and assume that the placement error of different holes is independent. By implementing it with *numpyro.plate*, we get i.i.d. samples.

$$\begin{aligned} N &\sim \text{Uniform}(340, 370) \\ r &\sim \text{Uniform}(50, 100) \\ \sigma &\sim \text{Uniform}(0, 1) \\ x_0 &\sim \text{Uniform}(0, 150) \\ y_0 &\sim \text{Uniform}(50, 200) \\ \alpha_j &\sim \text{Uniform}(-180, 180) \end{aligned}$$

Figure 2: Prior parameter Distributions for the Model based on Budiselic et al. Uniform prior is uninformative and does not favor any specific parameter value, reflecting our state of belief or knowledge.

If we disregard the data for a moment, our model will produce a Prior Predictive Distribution as shown in 3.[4] As we can see, our data is contained within this distribution, so the parameters that generate this data can be recovered and our prior ranges are reasonable.

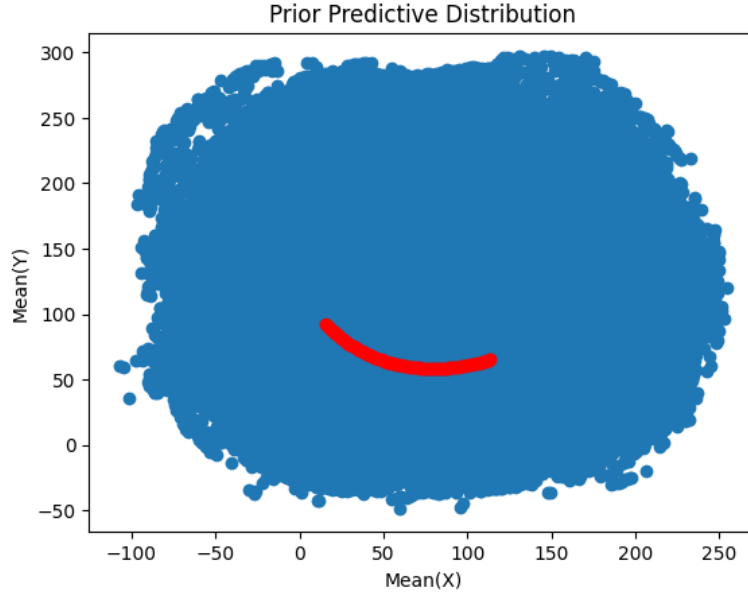


Figure 3: Visualization of Prior Predictive Distribution (blue) of our model based on 1000 samples. Illustrates that the real data (in red) is part of this distribution, hence the parameters generating the data can be recovered.

4 Model complication/extension

Instead of choosing a single sigma, we can follow the approach in [1] which differentiates between radial and tangential errors, adding one more free parameter to our model. It is not too big of an assumption to think that the Antikythera Mechanism was once a precisely scribed circle.[1] Therefore, "it is appropriate to differentiate between errors in the radial locations of the holes, and errors around the ring, which we will regard as tangential position errors".[1] Furthermore, it makes sense to take these as orthogonal vectors, or independent errors, since we expect them to be uncorrelated as well as assuming that σ_r and σ_t are the same for every hole and that the hole-to-hole errors are uncorrelated. [1] To incorporate this change into our model, we use a diagonal covariance matrix with distinct standard deviations for the radial, σ_r , and tangential, σ_t , directions: $\begin{bmatrix} \sigma_r & 0 \\ 0 & \sigma_t \end{bmatrix}$.

We note, however, that we slightly change our approach as compared to the simpler model: we are still using numpyro and the same logic, but now we need to calculate the covariance matrix separately for each hole, since we want it to be aligned with radial and tangential directions of that particular hole. Therefore, we update our *holes.locations* function accordingly to take σ_r and σ_t as parameters and return both holes positions and covariances matrices. This allows the model to capture that errors are likely larger in the tangential direction (around the ring) and smaller in the radial direction, potentially leading to more accurate estimates of the model parameters.

Overall, the covariance matrix represents the uncertainty in the measured hole locations. It also captures the correlation between the errors in the x and y directions. Also, the radial-tangential model for the covariance matrix might a priori be considered a good model for the following reasons. Firstly, the sources of error that affected the radial and tangential directions differently. For example, the radial position of the holes might have been determined by a scribed circle, while the tangential position might have been determined by a person's precision. Also, the radial-tangential model is more flexible than the isotropic model, as it has two parameters σ_r and σ_t instead of one σ .

This model's Prior Predictive distribution doesn't differ substantially from one shown in 3 and, thus, is not shown here separately.

5 Maximum Likelihood Fit

For the simpler model, we use a Gaussian likelihood function for each hole location, assuming that the errors in the placement of different holes are independent. More specifically, we sample each points from Multivariate normal distribution with standard deviation σ in both, x and y, directions and mean given by the output of *hole_locations* function. To assess how likely it is to observe the data for a particular set of parameters, we calculate the log likelihood. For each hole, we use the *log_prob* function, which calculates the logarithm of

probability of observing the data points according to the multivariate normal distribution. We further calculate the negative log likelihood of a set of parameters for one whole sample (of 81 points) by negating the sum of individual log probabilities for each hole.

To obtain the derivative of the log likelihood, we utilize jax auto-differentiation, using *jax.grad()*, applied to the negative log likelihood function, which takes parameters of our model as inputs rather than sampling them during execution. This is required for *jax.grad()* to compute the gradient as it needs to track the parameters. We test derivative evaluation on a few sets of parameters, and then use *scipy.optimize* on the negative log likelihood function to find the maximum likelihood parameters, shown in 2. The plot of the MLE hole locations for this model is presented along the maximum likelihood estimations for the radial-tangential model and is shown in 4.

Parameter	N	r	σ	x_{0j}	y_{0j}	α_j
MLE value	339.79	73.99	0.12	76.15	130.42	-2.52

Table 2: Table illustrating the estimated maximum likelihood parameters of the isotropic model, rounded to 2 decimal places. For x_{0j} , y_{0j} , α_j , the mean value is given since these parameters aren't of interest.

We use the same approach to calculate the log likelihood and its derivative for the complex, or radial-tangential, model. We find the MLE parameters, as shown in 3, and plot the resulting hole locations in 4.

Parameter	N	r	σ_r	σ_t	x_{0j}	y_{0j}	α_j
MLE value	342.11	74.60	0.05	0.12	77.31	131.64	-2.53

Table 3: Table illustrating the estimated maximum likelihood parameters of the radial-tangential model, rounded to 2 decimal places. For x_{0j} , y_{0j} , α_j , the mean value is given since these parameters aren't of prime interest.

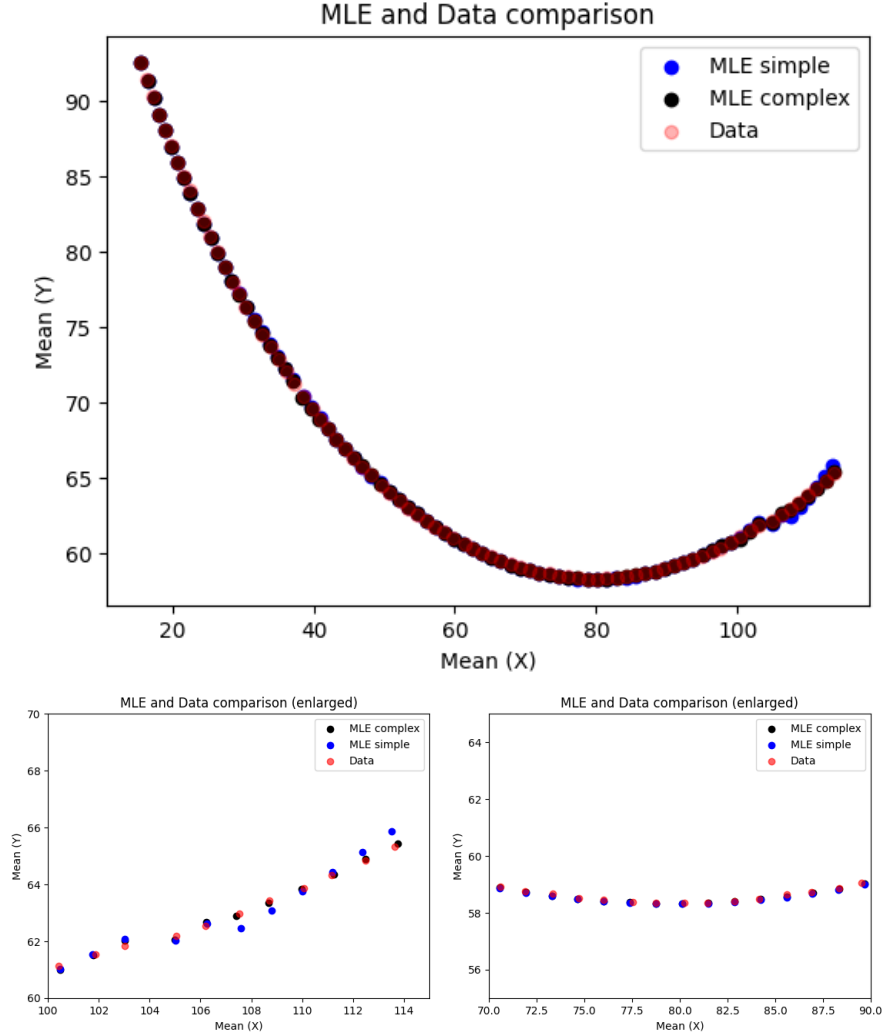


Figure 4: Hole locations from both models plotted using MLE of parameters in comparison to observed hole locations. Enlarged to visualize distinction. On the top plot: holes plotted with respective models MLE parameters in comparison to data, showcasing most visible distinction between models' 'predictions'. On the left bottom plot: both are shown with respect to observed data (in red) for last 11 holes. On the right bottom plot: both are shown with respect to observed data (in red) for holes 49-63, showcasing much less of a difference.

It is also possible to obtain likelihood values by using *log_likelihood* function from *numpyro.infer* module. Applying this function to every sample from our inference process (described in the next section), we get *log_likelihood* for every 81 point for each sample. To get a likelihood for each sample, we sum the *log_likelihoods* across 81 independent points (product of likelihoods is the sum of *log likelihoods*), and then find the maximum value. This gives us the Maximum Likelihood Estimation (MLE) for parameters, given our dataset and samples.

This limits, however, the accuracy as we are evaluating our `log_likelihood` and its derivative (using `jax.grad()`) on the samples which are discretely sampled from the continuous parameter space. To confirm the `log_likelihood` derivative extraction and calculation, we evaluate it for the first 1000 samples, obtained from the MCMC simulation which yields results without errors. Our plots of hole locations for such MLE estimates also showcase relatively good results, as shown in 5.

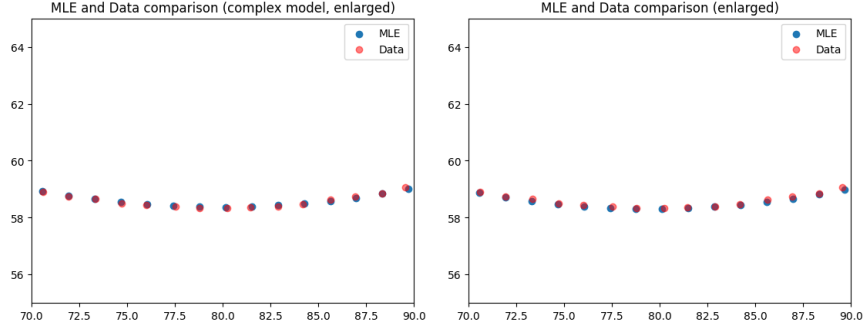


Figure 5: Hole locations from both models plotted using MLE of parameters in comparison to observed hole locations. Enlarged to visualize distinction. On the left plot: holes from model with isotropic covariances is shown with respect to observed data (in red) for holes 49-63. On the right plot: holes from model with radial and tangential covariances is shown with respect to observed data (in red) for holes 49-63.

6 Inference

We perform inference using numpyro’s internal inference engine, NUTS, No-U-Turn Sampler, an extension of Hamiltonian Monte Carlo (HMC) that eliminates the need for hand-tuning HMC’s parameters which can be both computationally expensive and time sensitive. HMC itself is a widely used Markov Chain Monte Carlo (MCMC) algorithm that can avoid random walk behavior (by using the gradient information of target distribution and equations of Hamiltonian mechanics), given it is well-tuned by the user. The most important parameters that need to be tuned are L , number of integration steps, and Δt , the time step. When Δt is too large - the Hamiltonian is not conserved due to inaccuracies of numerical integration which decreases acceptance probability. When L is too large, meaningless computations are made, increasing the computational cost of the algorithm, just like when both Δt and L are too small. NUTS performs this parameter tuning by itself using a recursive algorithm that cleverly stops from going backwards. [5] This helps avoid redundant sampling and allows it to adaptively find the posterior regions that represent the distribution of parameters more effectively. According to [5], NUTS performs as well as a well-tuned standard HMC, thus using NUTS does not compromise our results, but rather eliminates the need for hand-tuning the MCMC.

As traditional HMC methods, NUTS takes in a potential energy function,

or the negative log joint density for the model. [4] The numpyro internal inference engine calculates this potential energy function for us as well. We, first, initialize our MCMC kernel to use NUTS and apply it to our model, using: `nuts_kernel=numpyro.infer.NUTS(model)`. Then, we set up an MCMC inference, using our kernel and specifying number of warmup and sampling iterations: `mcmc = numpyro.infer.MCMC(nuts_kernel, num_warmup=5000, num_samples=10000)`. During the warmup stage, NUTS will tune the parameters, such as step size, to become more efficient during sampling. This stage is commonly referred to as burn-in stage which is described as the region where the chain remembers its starting position. Finally, to run our NUTS MCMC, we use `mcmc.run(subkey, data = data_points)` with inputting observed data. Creating a subkey using `key, subkey = jax.random.split(key)` is useful because we want to use different keys for different MCMC runs, and using them ensures reproducibility.

We note that there are some limitations in our implementation: we have not incorporated thinning which affects number of effective, or independent, samples and we only run the chain for 10,000 samples due to the issues with increasing number of chains for numpyro to use due to restrictions on CPU access. However, these limitations are not significant since effective number of samples is primarily needed to estimate the error - which means we just have relatively higher uncertainty.

7 MCMC Diagnostics

While there isn't a single quantitative test to perform that will confirm convergence of a Markov Chain, there are a few sanity checks which can suggest that, at least, nothing had gone completely wrong. First, we look at the trace plots of parameters, using ArviZ which integrates well with numpyro.[6] We want to see exploration by our MCMC of parameter space and whether it is learning centers and angular positions for each section as demonstrated in 7. Since we are using warmup in NUTS, we don't observe any burn-in on the trace plots.

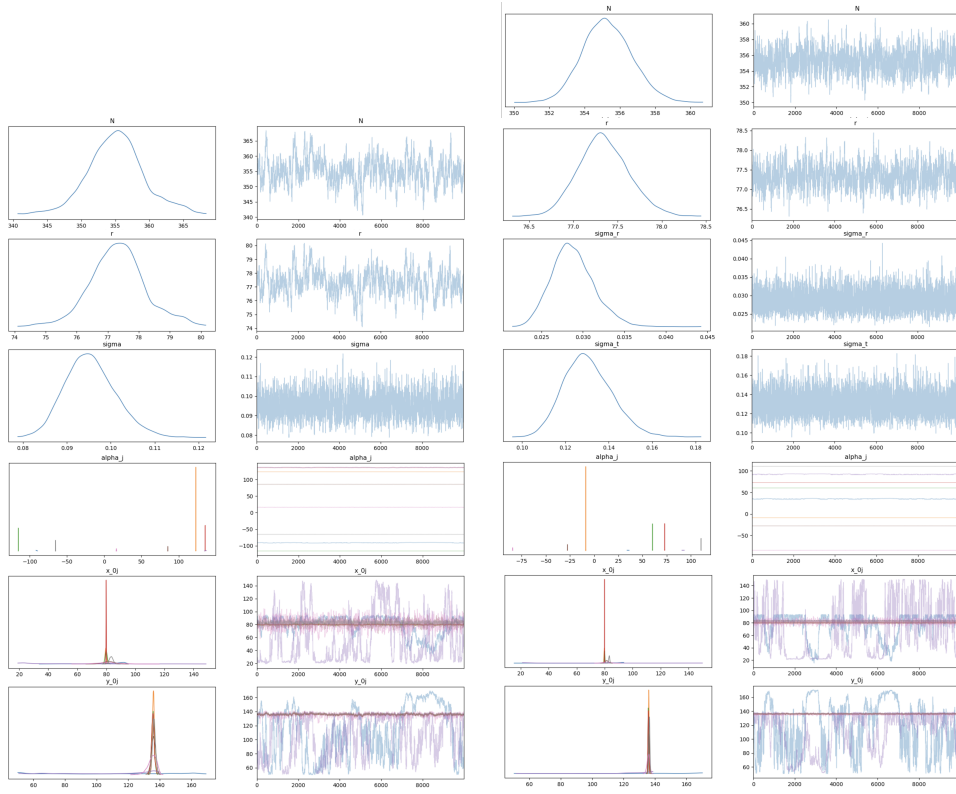


Figure 6: Trace plots of our inference engine, NUTS. Exhibiting parameter space exploration as expected. For our simpler model, on the left, the top plot is for N , then for r , and for σ . The next 3 plots show learning of specific α_j , x_{0j} and y_{0j} for each section by our model. On the right, for our complex model, the top plot is for N , then for r , for σ_r , and for σ_t . We also observe very similar behavior in the trace plots of α_j , x_{0j} and y_{0j} as for the other model. The two most wiggling traces in the last two plots refer to the 2 single-point arcs which can't constrain the parameters well.

Then, we look at histograms and corner plots which are illustrated in 7 and 8. We want to confirm that there is no unexpected behavior by assessing the shape of the posterior distribution and, since we know true parameters, we can plot them to check whether our model agrees with true values. As seen in 7 and 8, both our posteriors agree with the known 'true' (those, coming from [1]) values.

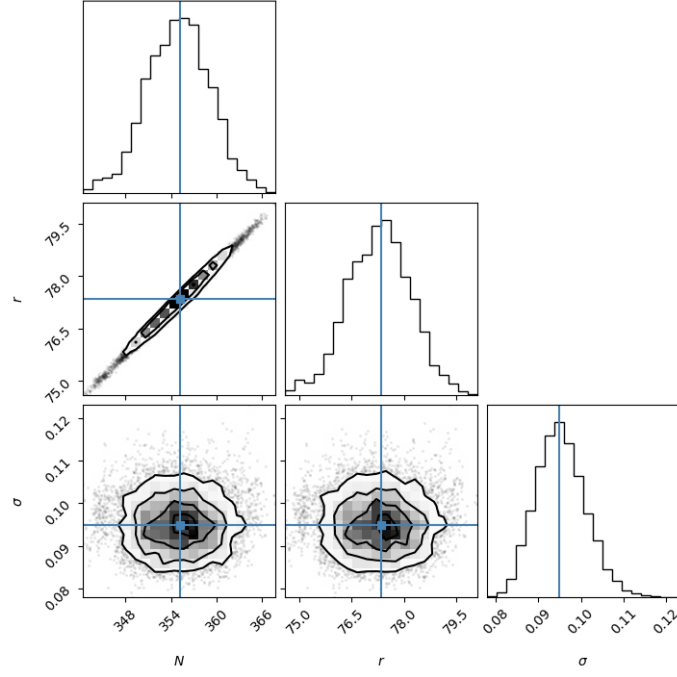


Figure 7: Corner plot of our learned Posterior Distribution for the simpler model. On the diagonals, marginal distributions for parameters N , r , and σ are illustrated. Off-diagonally, joint distributions of pairs of parameters are shown. Blue squares indicate the true values of parameters [1] which are shown to be in agreement with our model's output.

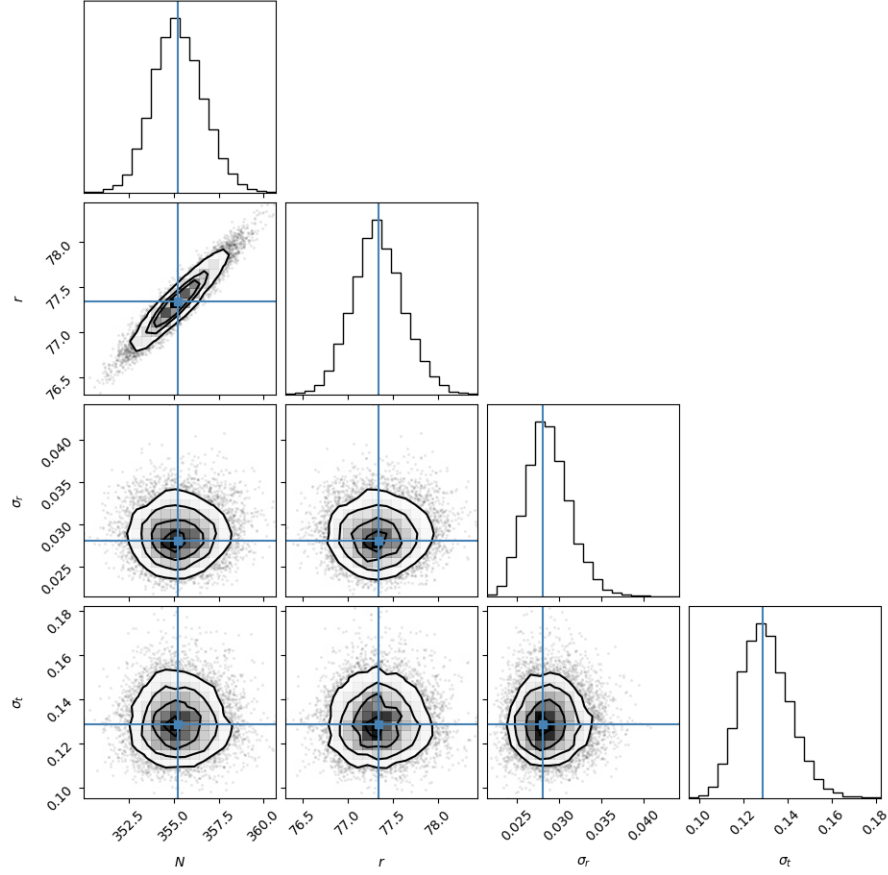


Figure 8: Corner plot of our learned Posterior Distribution for the more complex, or radial-tangential, model. On the diagonals, marginal distributions for parameters N , r , σ_r and σ_t are illustrated. Off-diagonally, joint distributions of pairs of parameters are shown. Blue squares indicate the true values of parameters [1] which are shown to be in agreement with our model's output.

Next, we check the $\hat{r}$ which is calculated internally by NUTS sampler. It is the GR statistic when multiple chains are run, but since we were only restricted to one chain it provides some measure of convergence of our chain. It is expected to be around 1 and that is seen in our MCMC summary for all parameters of both of our models, 9 and 10. Moreover, $\hat{r} \leq 1.2$ for all parameters which is a good indication of convergence.

	mean	std	median	5.0%	95.0%	n_eff	r_hat
N	355.15	4.27	355.12	347.93	362.31	108.37	1.00
alpha_j[0]	-90.81	0.55	-90.81	-91.67	-90.02	11.46	1.12
alpha_j[1]	123.13	0.01	123.14	123.13	123.14	1.15	1.00
alpha_j[2]	-115.62	0.01	-115.62	-115.64	-115.61	305.82	1.00
alpha_j[3]	135.71	0.01	135.71	135.69	135.72	24.48	1.00
alpha_j[4]	136.23	0.67	136.36	135.18	137.16	27.85	1.08
alpha_j[5]	85.42	0.05	85.42	85.33	85.50	527.79	1.00
alpha_j[6]	16.32	0.11	16.32	16.13	16.48	410.02	1.00
alpha_j[7]	-65.40	0.02	-65.40	-65.43	-65.36	193.12	1.00
r	77.32	0.92	77.31	75.83	78.93	107.63	1.00
sigma	0.10	0.01	0.09	0.09	0.10	1207.21	1.00
x_0j[0]	79.08	14.83	84.56	53.00	94.44	8.31	1.12
x_0j[1]	79.67	0.66	79.66	78.55	80.78	111.03	1.00
x_0j[2]	79.89	0.75	79.87	78.65	81.13	264.40	1.00
x_0j[3]	79.87	0.11	79.87	79.67	80.05	208.83	1.00
x_0j[4]	54.76	37.46	38.28	19.27	120.42	26.90	1.07
x_0j[5]	81.74	3.85	81.74	75.54	88.18	603.69	1.00
x_0j[6]	81.08	7.64	81.00	69.05	94.01	429.72	1.00
x_0j[7]	83.30	1.28	83.30	81.26	85.45	696.98	1.00
y_0j[0]	110.85	37.37	113.99	50.06	160.13	12.73	1.10
y_0j[1]	136.05	0.68	136.04	134.88	137.15	112.11	1.00
y_0j[2]	135.70	0.87	135.72	134.23	137.12	125.53	1.00
y_0j[3]	135.68	0.93	135.67	134.15	137.31	107.71	1.00
...							
y_0j[6]	135.23	2.81	135.57	131.08	139.65	357.71	1.00
y_0j[7]	136.42	0.97	136.40	134.86	138.10	143.55	1.00

Number of divergences: 0

Figure 9: Showing the summary of our MCMC process of the simpler model which was ran for 10,000 samples after 5,000 warm-up samples. n_eff shows number of efficient, or independent, samples and can be used to estimate auto-correlation time (IAT), $\hat{r}$ is a measure of convergence.

We note better accuracy of our radial-tangential (or more complex) model. While the isotropic model predicts $N = 355.15 \pm 4.27$ and $r = 77.32 \pm 0.92$, the radial-tangential model predicts $N = 355.26 \pm 1.40$ and $r = 77.34 \pm 0.29$.

	mean	std	median	5.0%	95.0%	n_eff	r_hat
N	355.26	1.40	355.21	352.98	357.56	458.14	1.00
alpha_j [0]	34.97	0.57	34.98	34.00	35.81	26.32	1.00
alpha_j [1]	-8.81	0.00	-8.81	-8.81	-8.81	930.92	1.00
alpha_j [2]	60.31	0.00	60.31	60.30	60.31	2.06	1.00
alpha_j [3]	72.88	0.00	72.88	72.87	72.88	361.83	1.00
alpha_j [4]	92.04	0.74	92.09	90.97	93.17	15.10	1.15
alpha_j [5]	-27.68	0.02	-27.68	-27.70	-27.65	1903.86	1.00
alpha_j [6]	-84.22	0.03	-84.22	-84.27	-84.16	1259.23	1.00
alpha_j [7]	110.54	0.01	110.54	110.52	110.55	505.97	1.00
r	77.34	0.29	77.33	76.88	77.82	416.19	1.00
sigma_r	0.03	0.00	0.03	0.02	0.03	2643.15	1.00
sigma_t	0.13	0.01	0.13	0.11	0.15	2955.91	1.00
x_0j [0]	76.11	19.57	83.82	41.15	93.53	21.18	1.00
x_0j [1]	79.69	0.20	79.69	79.35	80.01	428.92	1.00
x_0j [2]	79.91	0.23	79.91	79.53	80.28	1086.90	1.00
x_0j [3]	79.86	0.04	79.86	79.80	79.92	828.81	1.00
x_0j [4]	67.35	43.25	54.53	21.04	136.34	21.26	1.12
x_0j [5]	81.44	1.11	81.42	79.69	83.30	1953.78	1.00
x_0j [6]	81.44	2.51	81.47	77.14	85.41	1270.86	1.00
x_0j [7]	83.22	0.39	83.24	82.59	83.89	2264.96	1.00
y_0j [0]	117.46	36.32	123.20	63.32	170.18	30.56	1.00
y_0j [1]	136.04	0.21	136.03	135.68	136.37	429.95	1.00
y_0j [2]	135.72	0.27	135.71	135.26	136.16	475.30	1.00
...							
y_0j [6]	135.78	0.86	135.82	134.31	137.15	1039.72	1.00
y_0j [7]	136.42	0.30	136.42	135.96	136.95	520.32	1.00

Figure 10: Showing the summary of our MCMC process of the radial-tangential model which was ran for 10,000 samples after 5,000 warm-up samples. n_eff shows number of efficient, or independent, samples and can be used to estimate auto-correlation time (IAT), $\hat{r}$ is a measure of convergence.

Auto-correlation time is another statistic that should be checked. We can calculate it from the number of efficient samples, given in 9. For every 100 N and r samples, we only get one independent, or uncorrelated, sample which means that our IAT is quite large. This is reflected in comparison of the simpler model to the more complex model, as quadruple amount of effective samples for the radial-tangential model leads to much smaller error on standard deviation.

Altogether, these diagnostic checks collectively provide some confidence level of convergence behavior of our MCMC, helping to ensure that the MCMC methods result is meaningful and parameter estimates are accurate.

8 Model Comparison

To compare two models, evidence ratio is used in Bayesian Inference. In our case, we can avoid using Nested Sampling since we note that the more trivial model can be recovered from the more complex model by setting $\sigma_t = \sigma_r$. In such cases, Savage-Dickey ratio can be used to determine a preferred model, since our priors are consistent (setting $\sigma_t = \sigma_r$ leads to identical priors for both models), we can simply focus on the evidences. However, it is not necessary to perform additional calculations as we can see from the corner plot, the posterior of more complex model is peaked away from $\sigma_t = \sigma_r$ point which suggests that the more complex model is preferred.

Nevertheless, we proceed with calculating the Savage-Dickey ratio. We set $\sigma_t = \sigma_r$, and estimate the pdfs using *gaussian_kde*. We compute the ratio of evidences for to determine that the Savage-Dickey ratio is close to 0, meaning that the radial-tangential model is strongly preferred over the isotropic model. More specifically, we find an upper bound of Savage-Dickey ratio to be at most 0.00004 by estimating the minimum number of standard deviations $\sigma_t = \sigma_r$ is away from the peak of those distributions and approximating this probability with standard normal (sensible due to Central Limit theorem). In other words, the data provides strong evidence that the radial and tangential errors are different, and the isotropic model is not able to capture this difference.

9 Posterior Predictive Distribution

Looking at the Posterior Predictive Distribution is another sanity check after completing Bayesian Inference. Even though our selected model is better than other ones (in our case only other one model) - it doesn't mean that it is a good model. To check its suitability, we perform a Posterior Predictive Check by looking at the Posterior Predictive Distribution of our preferred model as shown in 11. We can also visualize Posterior Predictive Distribution of both models for a specific hole as shown in 12 to get a better understanding of what they actually look like at a larger scale. We conclude that our better model, the radial-tangential one, indeed captures the essence of the task and outperforms the isotropic covariances model.

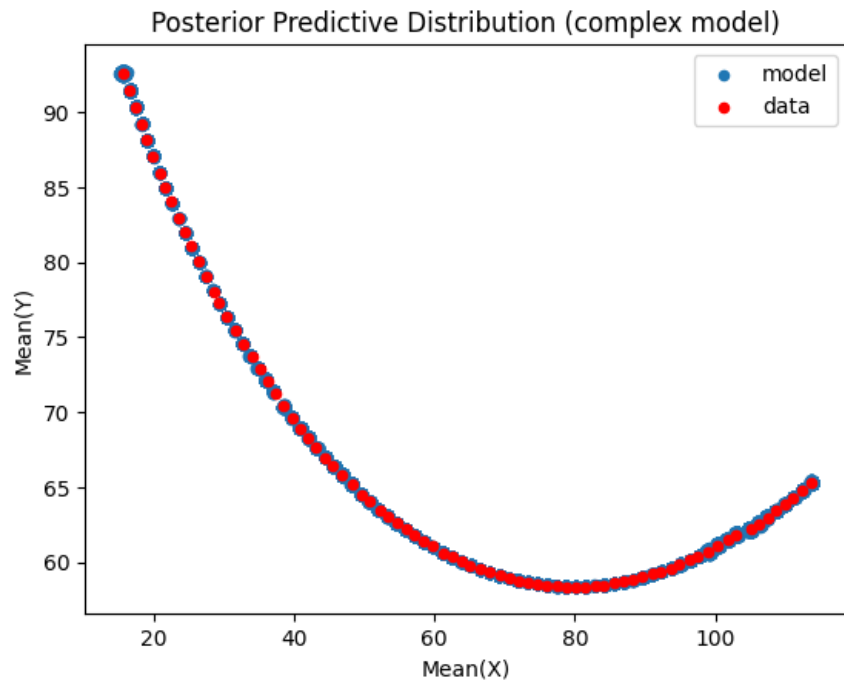


Figure 11: Showing the Posterior Predictive Distribution of our preferred model (complex) for all 81 points. From this point of view, it does appear to capture the essence of the problem. For this plot, the data points were slightly decreased in size for better visibility of model's posterior predictive distribution.

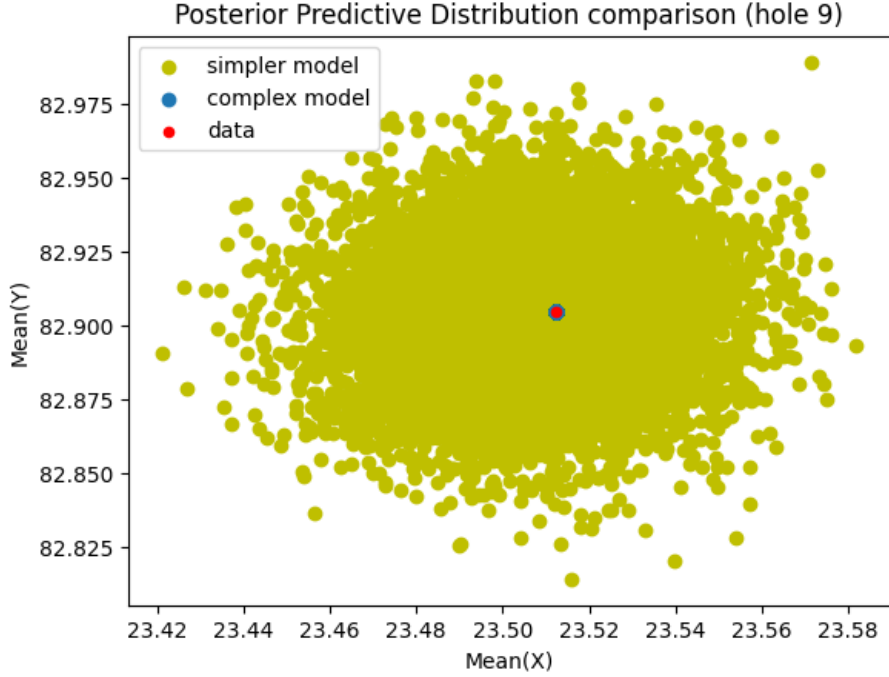


Figure 12: Showing the Posterior Predictive Distribution of both models for randomly chosen hole 9. As we can see, their shape differs drastically. The simpler model, with isotropic covariance, captures the data points, but has a drastic variability. On the other hand, the radial-tangential model exactly pin points the hole location. For this plot, the data points were slightly decreased in size to illustrate the blue point, the complex model prediction.

10 Further Possible Steps

As noted in [1], this analysis can be improved by leveraging the assumption of points being in (x,y) -plane and considering vertical displacement as well. For such an analysis, however, to be possible, a 3-D measurement of the remaining part of Antikythera mechanism is required. Perhaps, inter-hole distances can also be used to improve the bounds on N : by constraining the possible hole displacement from each other which might also help minimize the effect of cracks and separations of segments. Nonetheless, adding parameters might only refine this analysis, not undermine it. [1]

11 Summary

In this coursework, we performed Bayesian Inference with Hamiltonian Monte Carlo using a Probabilistic Programming Language, numpyro, using the x-Ray data from previous studies of the Antikythera Mechanism [3]. As found during

our analysis, the model that differentiates between radial and tangential sources of error outperforms the model that uses isotropic Gaussian likelihood for hole locations. Also, our refined estimate for the number of holes in the complete calendar ring is $N = 355.26 \pm 1.40$ (68% confidence interval). This estimate of the number of holes, along with its confidence interval, can assist archaeologists and other people interested in understanding the purpose of Antikythera Mechanism.

References

- [1] Graham Woan and Joseph Bayley. *An improved calendar ring hole-count for the Antikythera mechanism*. 2024. arXiv: [2403.00040](https://arxiv.org/abs/2403.00040) [physics.hist-ph]. URL: <https://arxiv.org/abs/2403.00040>.
- [2] Andrew Thoeni, Chris Budiselic, and Andrew Ramsey. *Replication Data for: Antikythera Mechanism Shows Evidence of Lunar Calendar*. Version V3. UNF:6:s5h5VJR6XwpEzLqKWKO68g== [fileUNF]. 2019. DOI: [10.7910/DVN/VJGLVS](https://doi.org/10.7910/DVN/VJGLVS). URL: <https://doi.org/10.7910/DVN/VJGLVS>.
- [3] C. Budiselic et al. “Antikythera Mechanism Shows Evidence of Lunar Calendar”. In: *The Horological Journal* (Mar. 2021). Accessed: 2025-03-21. URL: <https://doi.org/10.31235/osf.io/fzp8u>.
- [4] Pyro Team. *Bayesian Regression*. Accessed: 2025-03-21. 2023. URL: https://num.pyro.ai/en/latest/tutorials/bayesian_regression.html#Prior-Predictive-Distribution.
- [5] Matthew D. Hoffman and Andrew Gelman. *The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo*. 2011. arXiv: [1111.4246](https://arxiv.org/abs/1111.4246) [stat.CO]. URL: <https://arxiv.org/abs/1111.4246>.
- [6] Pyro Team. *Bayesian Hierarchical Linear Regression*. https://num.pyro.ai/en/latest/tutorials/bayesian_hierarchical_linear_regression.html#3.-Fitting-the-model. [Online; accessed 21-Mar-2025]. 2025.

A Appendix

ChatGPT was used to help with formatting of this LATEX document, specifically with the following tasks:

- item list creation
- image formatting
- centering an image that’s wider than text
- creation of references.bib file and formatting citations
- making a matrix in LATEX

- using tikz package
- using subfigure package, for Figure 6
- making a table